Design and Analysis of Algorithms

Dijkstra's Shortest Path Algorithm



Finding the Shortest Path in a Weighted Graph

A greedy approach that builds the answer from the closest known vertex.

Anika Mohite · PRN: 25070521184 · Section: D · 3rd Semester · Computer Science Engineering



What is Dijkstra's Algorithm?

Dijkstra's algorithm is a shortest-path algorithm used to find the minimum distance from a single source vertex to other vertices in a weighted graph.

- Finds shortest paths from one source vertex
- Works on weighted graphs
- Edge weights must be non-negative
- Repeatedly chooses the closest unvisited vertex
- Updates distances when a shorter route is found

Think of GPS navigation finding the shortest route from one location to another.

```
At each step, the best known distance becomes more reliable.
```

SOURCE → DISTANCE → SHORTEST PATH

The Weighted Graph

Vertices: A, B, C, D, E

A-B = 4

A-C = 2

C-B = 1

B-D = 5

C-D = 8

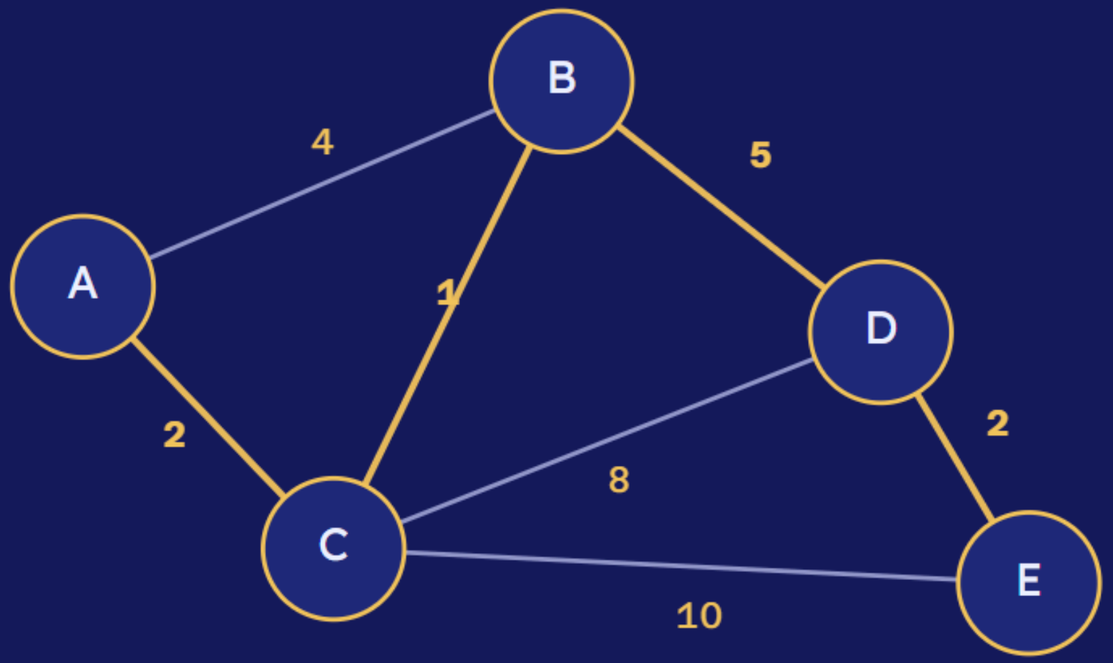
C-E = 10

D-E = 2

Source = A · Destination = E

Can we find the shortest path from A to E?

The edge weights represent the cost of moving between connected vertices.



How Does Dijkstra's Algorithm Work?



STEP 1 — INITIALIZE:

Set the source distance to 0. Set all other distances to infinity (∞).



STEP 3 — RELAX:

Check its neighbouring vertices. Update their distances if a shorter route is found.

`PICK → RELAX → MARK → REPEAT`



STEP 2 — SELECT:

Choose the unvisited vertex with the smallest known distance.



STEP 4 — REPEAT:

Mark the selected vertex as visited and continue. Once selected, its shortest distance is finalized.

Dijkstra Step-by-Step

Distance table · smaller estimates replace larger ones

Step	Selected	A	B	C	D	E
Start	—	0	∞	∞	∞	∞
1	A	0	4	2	∞	∞
2	C	0	3	2	10	12
3	B	0	3	2	8	12
4	D	0	3	2	8	10
5	E	0	3	2	8	10

From A: B = 4, C = 2

Choose C because 2 is smaller.

Through C: A → C → B = 2 + 1 = 3; A → C → D = 2 + 8 = 10; A → C → E = 2 + 10 = 12

Choose B because its distance is 3.

Through B: A → C → B → D = 3 + 5 = 8

Choose D.

Through D: A → C → B → D → E = 8 + 2 = 10

Therefore the shortest distance from A to E is 10.

Shortest Path Found!

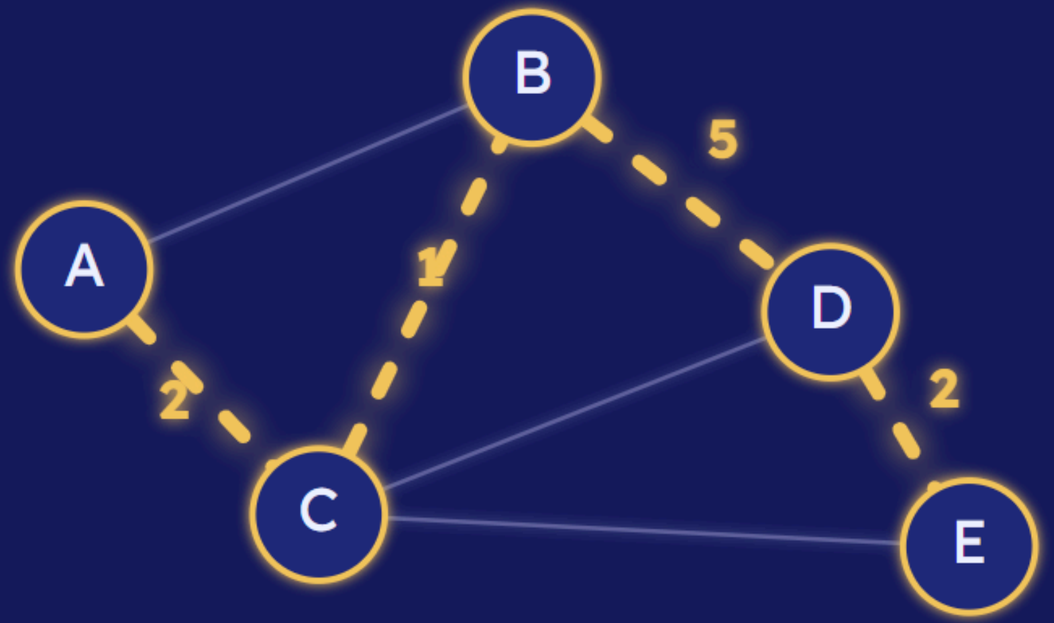
A → C → B → D → E

A → C = 2
C → B = 1
B → D = 5
D → E = 2

`TOTAL = 2 + 1 + 5 + 2 = 10`

`Shortest Distance = 10`

Dijkstra chooses the locally closest unvisited vertex and progressively builds the shortest-path solution. The final total is the sum of the selected edge weights.



Dijkstra's Algorithm — Pseudocode

```
Dijkstra(G, source)
```

1. Set dist[source] = 0
2. Set dist[all other vertices] = ∞
3. Mark all vertices as unvisited
4. Select the unvisited vertex u with minimum dist[u]
5. Mark u as visited
6. For every neighbour v of u: If dist[u] + weight(u,v) < dist[v]: Update dist[v]
7. Repeat until required vertices are processed
8. Return the shortest distances

```
KEY IDEA: If a newly discovered route is shorter, update the distance. This comparison is called relaxation.
```

SELECT → RELAX → VISIT → REPEAT

Complexity & Real-World Applications

TIME COMPLEXITY

Simple implementation: $O(V^2)$

Priority Queue / Min Heap: $O((V + E) \log V)$

V = number of vertices · E = number of edges

VIVA QUESTION

Can Dijkstra's algorithm work correctly with negative edge weights?

No. Dijkstra's algorithm assumes non-negative edge weights. For graphs with negative weights, algorithms such as Bellman-Ford are more appropriate.

APPLICATIONS



GPS Navigation → Finds efficient routes



Computer Networks → Finds minimum-cost paths



Robotics → Helps robots plan routes



Video Games → Character/path movement



Logistics → Efficient delivery routing

THANK YOU

Dijkstra's Shortest Path Algorithm

"Finding the shortest path, one step at a time."

Key takeaway: choose, relax, and repeat until the destination is reached.

Anika Mohite · PRN: 25070521184 · Section: D · 3rd Semester · Computer Science Engineering

Design and Analysis of Algorithms

